Experiment 30: Generative Model Output Comparison and Evaluation

Question

Problem Statement: The retailer's management must choose one generative model for production. They need objective evidence of which produces better and more varied images. The VAE and GAN outputs from Experiment 29 on Fashion-MNIST. The objective is to evaluate and compare generative models with quantitative and qualitative measures. Use TensorFlow, Keras, NumPy, Pandas, Scikit-learn, Matplotlib.

Tasks:

1. Generate 1,000 samples from each model.
2. Compute a classifier-based Inception Score and FID using a classifier trained on Fashion-MNIST.
3. Measure diversity and check for mode collapse through the class distribution of samples.
4. Compare real and generated feature distributions with t-SNE or PCA plots.
5. Tabulate all metrics with Pandas and write a recommendation.

Code

```
"""Experiment 30: Generative Model Output Comparison and Evaluation (VAE vs GAN)
Run Experiment 29 first: it saves vae_decoder.keras and gan_generator.keras in the working folder."""
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from scipy import linalg
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE

np.random.seed(42); tf.random.set_seed(42)
N = 1000
LATENT_VAE, LATENT_GAN = 2, 100

def load_fashion():
    try:
        (x, y), (xt, yt) = keras.datasets.fashion_mnist.load_data()
        return x[..., None] / 255.0, y, xt[..., None] / 255.0, yt
    except Exception as e:
        print("WARNING: Fashion-MNIST not available (", type(e).__name__, ") -> using scikit-learn digits as
a stand-in.")
        from sklearn.datasets import load_digits
        from scipy.ndimage import zoom
        d = load_digits(); im = np.clip(np.stack([zoom(i / 16.0, 3.5, order=1) for i in d.images]), 0,
1)[..., None]
        cut = int(0.75 * len(im)); return im[:cut], d.target[:cut], im[cut:], d.target[cut:]

x_train, y_train, x_test, y_test = [a.astype("float32") if a.dtype.kind == "f" else a for a in
load_fashion()]

# ---------------- Task 1: generate 1,000 samples from each model ----------------
try:
    decoder = keras.models.load_model("vae_decoder.keras"); generator =
keras.models.load_model("gan_generator.keras")
except Exception as e:
    raise SystemExit("Could not load vae_decoder.keras / gan_generator.keras - run Experiment 29 first in
this folder.")
vae_samples = decoder(tf.random.normal((N, LATENT_VAE))).numpy()
gan_samples = (generator(tf.random.normal((N, LATENT_GAN)), training=False).numpy() + 1) / 2
real_samples = x_test[:N]
sets = {"Real": real_samples, "VAE": vae_samples, "GAN": gan_samples}

# ---------------- Task 2: classifier-based Inception Score and FID ----------------
clf_in = keras.Input((28, 28, 1))
h = layers.Conv2D(32, 3, activation="relu")(clf_in); h = layers.MaxPooling2D()(h)
h = layers.Conv2D(64, 3, activation="relu")(h); h = layers.MaxPooling2D()(h)
feat = layers.Dense(64, activation="relu", name="features")(layers.Flatten()(h))
clf_out = layers.Dense(10, activation="softmax")(feat)
```

```
classifier = keras.Model(clf_in, clf_out)
classifier.compile(optimizer="adam", loss="sparse_categorical_crossentropy", metrics=["accuracy"])
classifier.fit(x_train, y_train, epochs=4, batch_size=128, verbose=0)
print(f"Classifier test accuracy: {classifier.evaluate(x_test, y_test, verbose=0)[1]:.4f}")
feature_extractor = keras.Model(clf_in, feat)


def inception_score(probs, eps=1e-12):
    py = probs.mean(axis=0, keepdims=True)
    return float(np.exp(np.mean(np.sum(probs * (np.log(probs + eps) - np.log(py + eps)), axis=1))))


def fid(f1, f2):
    mu1, mu2 = f1.mean(0), f2.mean(0); s1, s2 = np.cov(f1, rowvar=False), np.cov(f2, rowvar=False)
    s1 = s1 + 1e-6 * np.eye(len(s1)); s2 = s2 + 1e-6 * np.eye(len(s2))       # numerical stability
    covmean = linalg.sqrtm(s1 @ s2)
    if np.iscomplexobj(covmean): covmean = covmean.real
    return float(np.sum((mu1 - mu2) ** 2) + np.trace(s1 + s2 - 2 * covmean))


probs = {k: classifier.predict(v, verbose=0) for k, v in sets.items()}
feats = {k: feature_extractor.predict(v, verbose=0) for k, v in sets.items()}
# Reference real set for FID: use a DIFFERENT 1,000 real images from the test set (so Real-vs-Real is a
sensible baseline)
ref_feats = feature_extractor.predict(x_test[N:2 * N] if len(x_test) >= 2 * N else x_train[:N], verbose=0)


# ---------------- Task 3: diversity and mode collapse ----------------
rows = []
for k in sets:
    pred = probs[k].argmax(axis=1)
    dist = np.bincount(pred, minlength=10) / len(pred)
    ent = -np.sum(dist[dist > 0] * np.log(dist[dist > 0])) / np.log(10)          # 1.0 = perfectly uniform
over 10 classes
    idx = np.random.choice(len(sets[k]), 200, replace=False); flat = sets[k][idx].reshape(200, -1)
    pair = np.mean(np.linalg.norm(flat[:, None] - flat[None], axis=2))
    rows.append({"Model": k, "Inception Score": inception_score(probs[k]), "FID (lower=better)":
fid(feats[k], ref_feats),
                 "Class entropy (0-1)": ent, "Classes covered": int((dist > 0.01).sum()), "Max class share":
dist.max(),
                 "Mean pairwise distance": pair, "Mean confidence": probs[k].max(axis=1).mean()})


fig, ax = plt.subplots(1, 3, figsize=(15, 3.8), sharey=True)
for a, k in zip(ax, sets):
    a.bar(range(10), np.bincount(probs[k].argmax(axis=1), minlength=10) / len(probs[k])); a.set_title(f"{k}:
predicted class distribution")
    a.set_xlabel("Class"); a.axhline(0.1, color="r", ls="--", lw=1)
ax[0].set_ylabel("Fraction of samples"); plt.tight_layout(); plt.show()


# ---------------- Task 4: real vs generated feature distributions ----------------
M = min(500, len(real_samples))
allf = np.vstack([feats[k][:M] for k in sets]); labels = np.repeat(list(sets), M)
fig, ax = plt.subplots(1, 2, figsize=(13, 5.5))
for a, emb, title in zip(ax, [PCA(2).fit_transform(allf), TSNE(2, init="pca",
random_state=0).fit_transform(allf)], ["PCA of classifier features", "t-SNE of classifier features"]):
    for k, c in zip(sets, ["tab:blue", "tab:orange", "tab:green"]):
        m = labels == k; a.scatter(emb[m, 0], emb[m, 1], s=6, alpha=0.5, c=c, label=k)
    a.set_title(title); a.legend(markerscale=3)
plt.tight_layout(); plt.show()

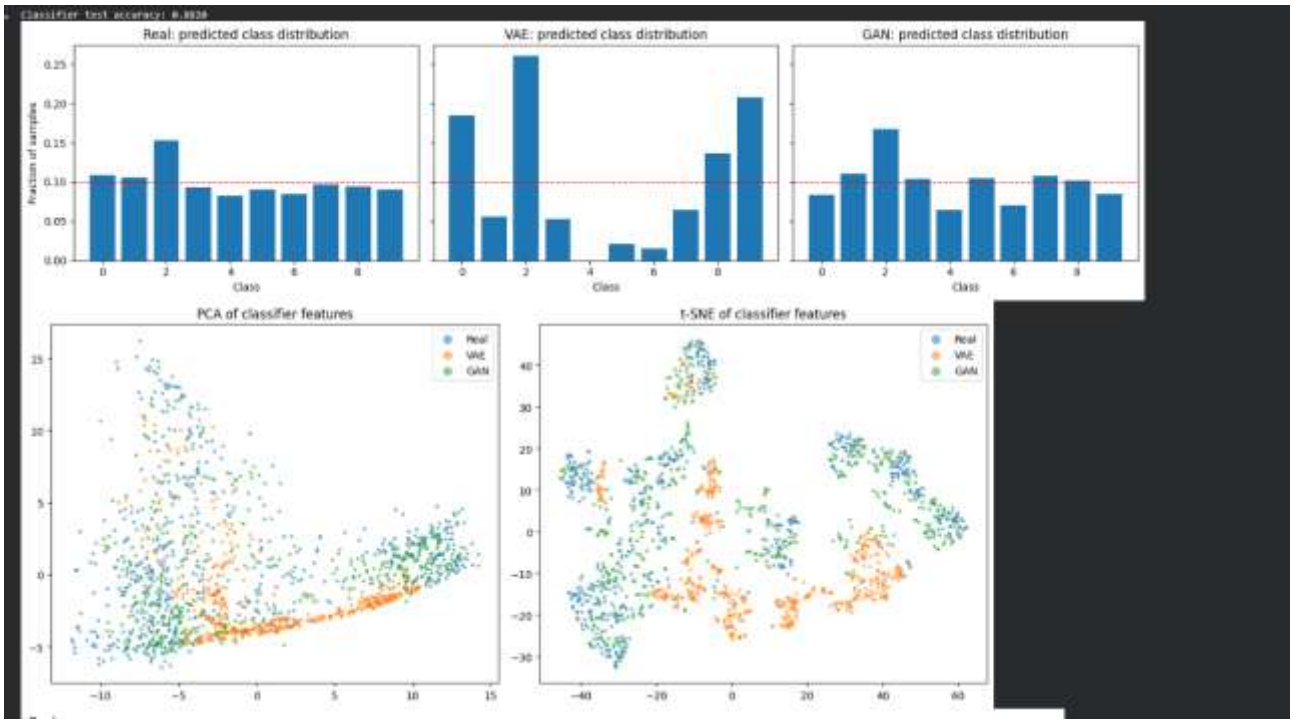

fig, axes = plt.subplots(3, 10, figsize=(14, 4.5))
for r, k in enumerate(sets):
    for c in range(10):
        axes[r, c].imshow(sets[k][c, :, :, 0], cmap="gray"); axes[r, c].axis("off")
    axes[r, 0].set_title(k, loc="left")
plt.tight_layout(); plt.show()


# ---------------- Task 5: table and recommendation ----------------
table = pd.DataFrame(rows).set_index("Model")
pd.set_option("display.width", 220); pd.set_option("display.max_columns", None)
print("\n", table.round(3))
table.to_csv("generative_model_metrics.csv")


cand = table.loc[["VAE", "GAN"]]
best_fid = cand["FID (lower=better)"].idxmin(); best_is = cand["Inception Score"].idxmax(); best_div =
cand["Class entropy (0-1)"].idxmax()
print(f"\nBest FID: {best_fid} | best Inception Score: {best_is} | most diverse (class entropy): {best_div}")
```

```
points = pd.Series({"VAE": 0, "GAN": 0}, dtype=float)
points[best_fid] += 2; points[best_is] += 1; points[best_div] += 1            # FID counts double
rec = best_fid if points.nunique() == 1 else points.idxmax()     # tie -> lowest FID
print(f"Points (FID x2, IS x1, diversity x1): {points.to_dict()}")
print(f"RECOMMENDATION: {rec}.")
print("Rationale: FID compares the whole feature distribution to real data (most important); Inception Score
rewards confident, "
      "varied samples; class entropy and the class histogram reveal mode collapse (a few classes dominating).
"
      "A model with low FID but low class entropy should be treated with caution.")
```

Output





[Paste output / graph screenshots here]